# 🎓 StudentHub — Engineering Report

Complete Codebase Architecture, Mathematical Algorithms & Team Presentation Guide

**Course:** 1st Year B.Tech Web Development **Platform:** StudentHub (Offline-First)

**Stack:** Semantic HTML5, Vanilla CSS3, JavaScript ES6+ **Storage:** HTML5 localStorage + Native Node.js Server

## 🎓 StudentHub — Complete Codebase Architecture & Team Walkthrough Guide

> **Target Audience**: Team Members, Project Partners, Code Reviewers, and Evaluators / Faculty.
> **Purpose**: A comprehensive, step-by-step technical breakdown of **how StudentHub works**, line-by-line architectural logic, mathematical algorithms, data schemas, security precautions, and a viva/presentation Q&A cheat sheet.

---

### 📑 Table of Contents


1. Project Overview & Core Philosophy
2. High-Level System Architecture & Directory Map
3. Data Flow, State Management & LocalStorage Schema
4. File-by-File Technical Deep Dive
   - 4.1 Global Styles & Design System ( `css/style.css` )
   - 4.2 Storage Abstraction & Date Engine ( `js/storage.js` )
   - 4.3 Common Utilities, Theme & Audio ( `js/common.js` )
   - 4.4 Executive Dashboard ( `index.html` & `js/dashboard.js` )
   - 4.5 Exam Planner & Study Scheduler ( `planner.html` & `js/planner.js` )
   - 4.6 Budget Tracker & CSS Charting ( `budget.html` & `js/budget.js` )
   - 4.7 Attendance Calculator & Smart Hints ( `attendance.html` & `js/attendance.js` )
   - 4.8 To-Do List & Pomodoro Timer Engine ( `todo.html` & `js/todo.js` )
   - 4.9 Automated In-Browser Test Suite ( `js/tests.js` )
   - 4.10 Optional Zero-Dependency Backend ( `server.js` )
   - 4.11 End-to-End Headless Verification ( `verify_e2e.js` )
5. Core Mathematical Algorithms & Complexity Analysis
6. Security, Performance & Edge-Case Handling
7. Team Presentation & Viva Q&A Guide
8. Live Demonstration Script (3-Minute Walkthrough)


---

## 1. Project Overview & Core Philosophy

### What is StudentHub?

**StudentHub** is an offline-first, all-in-one productivity suite specifically engineered for first-year Indian college and university students. During their transition into university, students typically struggle with cognitive fragmentation caused by using separate disconnected apps:

- Notion / physical diaries for syllabus schedules.
- Splitwise / Excel for canteen and hostel allowances.
- College ERP portals or WhatsApp notes for calculating attendance bunks.
- Distracting commercial to-do apps with ads, sign-in walls, and subscription popups.

StudentHub brings all five essential workflows into one cohesive, distraction-free environment:

1. **Executive Dashboard**: Centralized high-level summary cards.
2. **Exam Planner**: Day-wise adaptive study schedule generator.
3. **Budget Tracker**: Daily safe-to-spend allowance calculator with pure CSS visualization.
4. **Attendance Tracker**: Class bunk/recovery calculator based on strict institutional cutoffs (e.g., 75%).
5. **To-Do + Pomodoro**: Two-tier prioritized tasks integrated with a Web Audio API study timer.

### Architectural Philosophy

1. **Zero External Runtime Dependencies**: No React, Vue, Angular, Tailwind, Bootstrap, or jQuery. Built exclusively with **Semantic HTML5, Vanilla CSS3, and ES6+ JavaScript**.
2. **Offline-First & Privacy-Preserving**: All student records stay strictly inside the student's browser via HTML5 `localStorage`. No cloud tracking, no cookies, no mandatory login, and no telemetry.
3. **Dual-Mode Capability**:
   - **Mode A (Static / Browser-Only)**: Double-click `index.html` or open via any static hosting (GitHub Pages, Vercel, Live Server). Works 100% offline.
   - **Mode B (Full-Stack Mode)**: Run `node server.js` using Node's native HTTP and File System modules for persistent file storage (`data/database.json`) and RESTful endpoints without installing any `node_modules`!

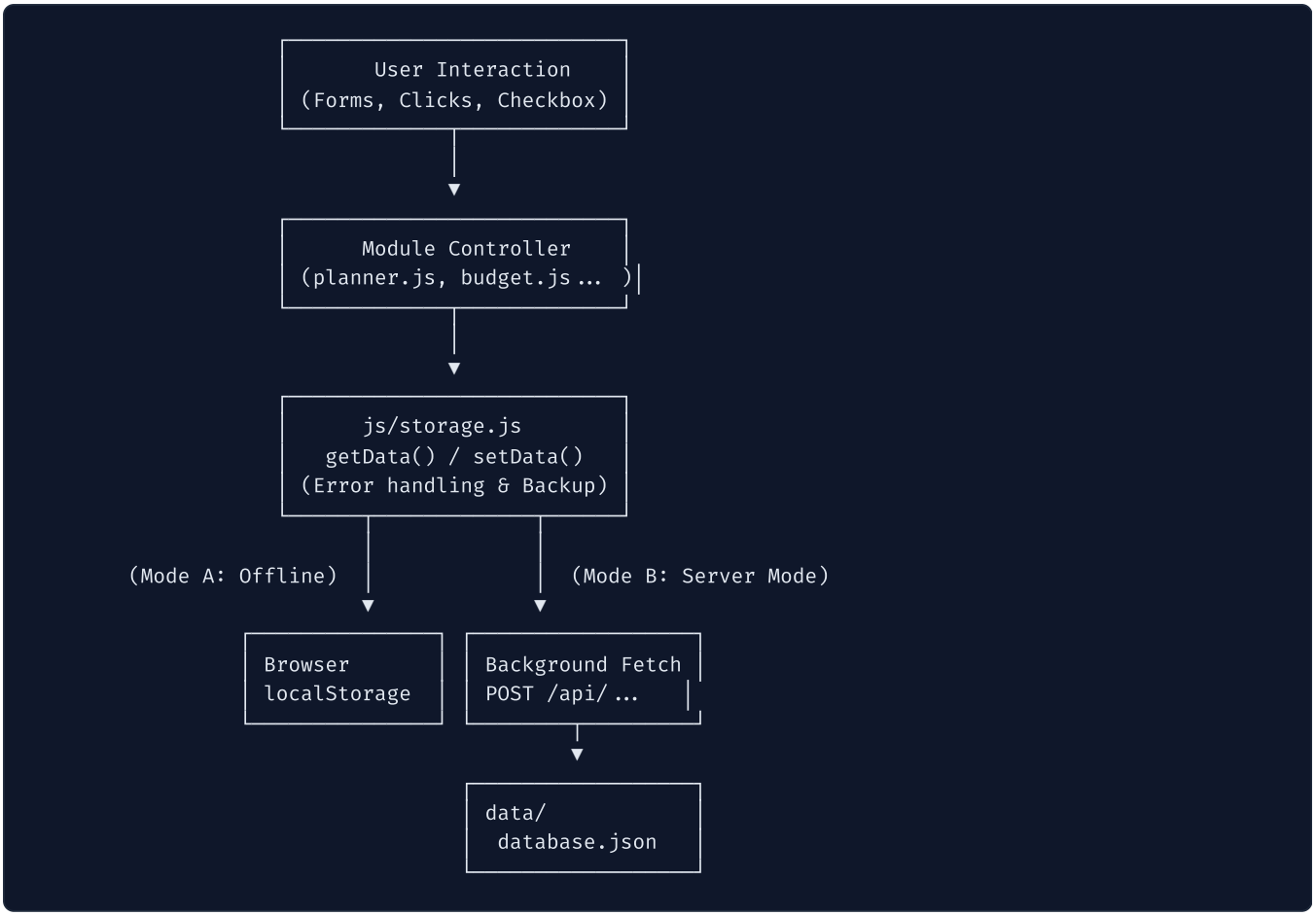
---

## 2. High-Level System Architecture & Directory Map

```
Project/
├── index.html                  # Root redirect entry point
├── studenthub/                 # Core application package
│   ├── index.html              # 1. Executive Dashboard & Unit Test Modal
│   ├── planner.html            # 2. Exam Planner & Study Scheduler
│   ├── budget.html             # 3. Monthly Budget Tracker & Expense Logger
│   ├── attendance.html         # 4. Attendance & Skip/Recovery Calculator
│   ├── todo.html               # 5. Prioritized Tasks & Pomodoro Timer
│   ├── package.json            # NPM metadata & start scripts (zero dependencies)
│   ├── server.js               # Built-in Node.js HTTP REST & Static Server
│   ├── verify_e2e.js           # Automated headless integration test suite
│   ├── data/
│   │   └── database.json       # Persistent JSON database (created in server mode)
│   ├── css/
│   │   └── style.css           # Unified Design System, CSS Variables & Themes
│   └── js/
│       ├── storage.js          # Storage layer, error boundaries & date math
│       ├── common.js           # Navbar, theme engine, toast, HTML sanitization
│       ├── dashboard.js        # Dashboard aggregation, demo data loader
│       ├── planner.js          # Exam syllabus tracker & study plan algorithm
│       ├── budget.js           # Budget engine, rollover & CSS bar chart
│       ├── attendance.js       # Attendance calculations & undo manager
│       ├── todo.js             # Task sorting & Web Audio Pomodoro engine
│       └── tests.js            # 10 automated browser-based unit tests
```

## 3. Data Flow, State Management & LocalStorage Schema

All client-side data is persisted in browser `localStorage`. Every module accesses data strictly through the abstraction layer defined in `js/storage.js`.


User Interaction
(Forms, Clicks, Checkbox)
Module Controller
(planner.js, budget.js ... )
js/storage.js
getData() / setData()
(Error handling & Backup)
(Mode A: Offline)
(Mode B: Server Mode)
Browser
localStorage
Background Fetch
POST /api/ ...
data/
database.json

### LocalStorage Schema Specifications

| Key | Type | Data Structure & Example | Description |
|---|---|---|---|
| `sh_subjects` | `Array<Object>` | `[ { "id": "sub_1", "name": "Data Structures", "examDate": "2026-11-15", "topics": [ { "id": "top_1", "title": "Binary Trees", "done": false } ] } ]` | Enrolled academic courses, exam dates (`YYYY-MM-DD`), and individual syllabus checklist topics. |
| `sh_budget` | `Object` | `{ "month": "2026-10", "allowance": 6000, "expenses": [ { "id": "exp_1", "amount": 250, "category": "Food", "date": "2026-10-24", "note": "Hostel mess fee" } ], "history": [ { "month": "2026-09", "allowance": 5000, "spent": 4820 } ] }` | Monthly allowance, list of categorized expenses, and archived past months. |
| `sh_attendance` | `Array<Object>` | `[ { "id": "att_1", "name": "Mathematics I", "required": 75, "present": 18, "total": 22, "lastMarked": "2026-10-24", "lastAction": "present" } ]` | Attendance tracking per course, required cutoff %, and last action pointer for undo operations. |
| `sh_todos` | `Array<Object>` | `[ { "id": "td_1", "text": "Submit Lab Assignment", "priority": "High", "due": "2026-10-28", "done": false } ]` | Task management with 3 priority tiers (`High`, `Medium`, `Low`) and optional deadline dates. |
| `sh_pomodoros` | `Object` | `{ "date": "2026-10-25", "count": 4 }` | Daily counter tracking completed 25-minute Pomodoro study intervals. |
| `sh_theme` | `String` | `"light"` or `"dark"` | User interface theme preference applied dynamically to the root HTML element. |

---

## 4. File-by-File Technical Deep Dive

---

### 4.1 Global Styles & Design System (`css/style.css`)

`style.css` contains the entire visual design system for all 5 pages. It avoids CSS bloat by defining systematic design tokens.

#### Key Architectural Components:

1. **CSS Custom Properties (Variables)**:
   - Root variables `:root` define Light Mode colors (`--bg-primary: #f8fafc;`, `--text-primary: #0f172a;`, `--card-bg: #ffffff;`, `--primary: #4f46e5;`).
   - `[data-theme="dark"]` overrides these tokens with slate-toned, low-strain night colors (`--bg-primary: #0f172a;`, `--card-bg: #1e293b;`, `--text-primary: #f1f5f9;`).
2. **Typography**:

- Primary heading font: **Poppins** (modern, bold geometric sans-serif).
- Body font: **Inter** (high legibility at small sizes for numbers and tables).
- System font fallbacks (`system-ui, -apple-system, sans-serif`) guarantee zero render blocking if offline.

3. **Pure CSS Responsive Layouts**:
   - Flexbox and CSS Grid layout engines.
   - Dynamic breakpoint at `768px` transforms the top navigation bar into a mobile slide-out hamburger menu.
4. **Visual Data Visualizations**:
   - **Progress Bars**: Pure CSS `<div class="progress-bar"><div class="progress-fill" style="width: X%;"></div></div>` with smooth CSS transitions.
   - **Proportional Stacked Bar Chart** (`.category-bar-chart`): Flexbox container where each child segment receives a proportional width (`width: 35%`) and distinct category color.
5. **Toast Notifications & Modals**:
   - Fixed-position `.toast-container` floating at the bottom-right corner with `@keyframes slideInUp`.
   - Accessible `.modal-backdrop` dialogs with backdrop blur (`backdrop-filter: blur(4px)`).

---

### 4.2 Storage Abstraction & Date Engine (`js/storage.js`)

`storage.js` acts as the safety gateway between raw browser storage and UI components.

#### Essential Functions:

1. `getData(key, fallback)`:
   - Wraps `localStorage.getItem()` in a `try ... catch` block.
   - Parses the JSON string; if parsing fails, or key does not exist, it safely returns the supplied `fallback` value, preventing application crashes.
2. `setData(key, value)`:
   - Serializes data via `JSON.stringify()`.
   - Intercepts `QuotaExceededError` (e.g., if mobile storage is exhausted) and displays a non-blocking toast warning to the user.
   - Automatically synchronizes with `server.js` if running in HTTP mode.
3. `calculateDaysLeft(fromDateStr, targetDateStr)`:
   - Normalizes both dates to **UTC Midnight** (`00:00:00.000Z`).
   - Prevents daylight savings and timezone conversion discrepancies.
   - Formula:

     $$\Delta days = round\left(\frac{targetMidnight - fromMidnight}{86,400,000 ms}\right)$$
4. `exportAllData()` & `importDataFromFile(file)`:
   - Allows students to export their entire productivity database as a single portable JSON file (`studenthub_backup_YYYY-MM-DD.json`).
   - Implements schema validation on imported JSON before overwriting storage to prevent file corruption.

---

### 4.3 Common Utilities, Theme & Audio (`js/common.js`)

`common.js` loads on every page and manages universal interface behaviors.

**Essential Features:**

1. **Theme Controller**:
   - `initTheme()` and `applyTheme(theme)`: Reads `sh_theme` from storage on initial page load to eliminate dark mode flashing. Toggles the `data-theme="dark"` attribute on `<html>`.
2. **Accessible Navbar & Active State**:
   - `highlightActiveNavLink()` inspects `window.location.pathname` and adds the `.active` class to the matching navigation link.
   - `initHamburgerMenu()` binds accessible `aria-expanded` attributes for mobile screen-readers.
3. **Toast Notification System**:
   - `showToast(message, type = 'info', duration = 3000)`: Dynamically generates alert elements with icons (`success`, `warning`, `error`, `info`), automatically unmounting them via `setTimeout`.
4. **Security Sanitizer (`escapeHTML`)**:
   - Prevents Stored Cross-Site Scripting (XSS).
   - Replaces `&`, `<`, `>`, `"`, `'` with HTML entities before injecting user input into `innerHTML`.
5. **Web Audio API Synthesizer (`playChime`)**:
   - Generates an acoustic chime without relying on external `.mp3` or `.wav` files.
   - Uses `window.AudioContext` to construct dual sine-wave audio oscillators:
     - Oscillator 1: **587.33 Hz** (D5 note)
     - Oscillator 2: **880.00 Hz** (A5 note)
   - Modulates an exponential decay envelope (`gainNode.gain.exponentialRampToValueAtTime`) for a gentle chime sound.

---

### 4.4 Executive Dashboard (`index.html` & `js/dashboard.js`)

The dashboard serves as the central cockpit, aggregating live metrics from all other four modules.

**Functional Logic:**

1. `renderGreeting()`:
   - Queries `new Date().getHours()` to dynamically display:
     - `Good morning` (05:00 - 11:59)
     - `Good afternoon` (12:00 - 16:59)
     - `Good evening` (17:00 - 04:59)
2. `renderExamCard()`:
   - Iterates through `sh_subjects`, finds the minimum positive `daysLeft`.
   - If `daysLeft ⩽ 3`, renders a glowing red warning badge (`badge-danger`).
3. `renderMoneyCard()`:
   - Reads `sh_budget`, computes total expenses for the active month, and calculates `remaining = allowance - spent`.
   - Displays safe-to-spend per day and percentage spent progress bar.
4. `renderAttendanceCard()`:
   - Iterates through `sh_attendance`, calculates individual percentages, and locates the subject with the lowest score.
   - Emphasizes low-attendance courses with urgent warning banners if below cutoff (e.g. 75%).

5. `renderTasksCard()`:
   - Filters pending tasks, sorts them using the two-tier algorithm (High Priority first, followed by earliest due date), and renders the top 3 items.
6. `loadDemoData()`:
   - Pre-populates realistic Indian college records (Data Structures, Engineering Physics, Canteen snacks, Hostel mess dues, High-priority assignments) with one click for easy evaluation.
7. `clearAllData()`:
   - Provides a double-confirmation modal before purging `localStorage`.

---

### 4.5 Exam Planner & Study Scheduler (`planner.html` & `js/planner.js`)

The Exam Planner manages academic subjects, topics, and auto-generates a balanced study schedule.

#### Key Mechanisms:

1. **Input Validation**:
   - `initExamDateMin()` enforces that the HTML `<input type="date">` minimum selectable date is `today`. Prevents adding exams in the past.
2. **Topic Management**:
   - Students can add granular topics under any subject.
   - Ticking off a topic instantly updates that subject's progress bar percentage:

$$\text{Progress } \% = \text{round}\left(\frac{\text{Completed Topics}}{\text{Total Topics}} \times 100\right)$$

3. **Adaptive Study Plan Algorithm**:
   - Spreads unfinished syllabus topics evenly across the days remaining before the exam date.
   - If a student marks a topic as done, or clicks the **"Reschedule"** button, the scheduler recomputes the distribution for remaining days.

---

### 4.6 Budget Tracker & CSS Charting (`budget.html` & `js/budget.js`)

Tracks college allowances, hostel expenses, and daily spending discipline.

#### Key Mechanisms:

1. **Allowance Setup & Safe Daily Limit**:
   - Allows students to configure their monthly allowance in Indian Rupees (₹).
   - Computes daily safe-to-spend:

$$\text{Daily Safe Limit} = \max\left(0, \left\lfloor \frac{\text{Remaining Allowance}}{\text{Days Left in Current Month}} \right\rfloor\right)$$

2. **Warning Banners**:
   - **Orange Warning Banner**: Displays when total expenditure reaches or exceeds **80%** of the monthly budget.
   - **Red Danger Banner**: Displays when total expenditure exceeds **100%** (budget overrun).
3. **Pure CSS Proportional Breakdown Chart**:
   - Groups expenses into standard categories: `Food`, `Travel`, `Recharge`, `Study`, `Fun`, `Other`.
   - Calculates percentage share per category.
   - Injects proportional flex items into a CSS container bar without using external charting libraries.

4. **Month Rollover & Archive History**:
   - `checkMonthRollover()` detects when the system calendar advances to a new month (e.g., from `2026-10` to `2026-11`).
   - Automatically archives the completed month's total into the `history` array and resets current expenses for the new month.

---

### 4.7 Attendance Calculator & Smart Hints (`attendance.html` & `js/attendance.js`)

Empowers students to maintain university attendance requirements (e.g. 75%) through actionable mathematical formulas.

#### Key Mechanisms:

1. **Attendance Logging**:
   - Quick one-tap **"Present"** and **"Absent"** buttons.
   - Increments `present` and `total` counters in `sh_attendance`.
2. **Undo Action Stack**:
   - Maintains `lastAction` metadata (`"present"` or `"absent"`).
   - Clicking **"Undo last"** safely reverts the previous entry, preventing accidental taps on mobile touchscreens.
3. **Smart Guidance Hints**:
   - **If current % `≥` required %**: Uses **Algorithm 3 (Skip Calculator)** to tell the student exactly how many upcoming classes can be safely skipped without dropping below cutoff:

     $$\text{Safe Skips} = \left\lfloor \frac{\text{Present} \times 100}{\text{Required}} - \text{Total} \right\rfloor$$

   - **If current % `<` required %**: Uses **Algorithm 4 (Recovery Calculator)** to tell the student exactly how many consecutive future classes must be attended to regain good standing:

     $$\text{Classes to Attend} = \left\lceil \frac{\text{Required} \times \text{Total} - 100 \times \text{Present}}{100 - \text{Required}} \right\rceil$$

---

### 4.8 To-Do List & Pomodoro Timer Engine (`todo.html` & `js/todo.js`)

Combines assignment tracking with an integrated Pomodoro study timer.

#### Key Mechanisms:

1. **Two-Tier Sorting Algorithm**:
   - Unfinished tasks are sorted first by priority level (`High` `\to` `Medium` `\to` `Low`).
   - Tasks with identical priority are sorted chronologically by due date (`earliest` first).
   - Tasks without a due date are placed after dated tasks.
   - Completed tasks are sorted to the bottom.
2. **Overdue Detection**:
   - Checks if `task.due < todayStr && !task.done`.
   - Attaches a prominent red `Overdue` badge to alert the student.
3. **Pomodoro Engine State Machine**:
   - Operates in two modes: **Focus (25 minutes)** and **Break (5 minutes)**.
   - Controlled via `startPomodoro()`, `pausePomodoro()`, and `resetPomodoro()`.

- Uses `setInterval()` ticking every second.
- Synchronizes remaining time to the browser tab title: `document.title = "(24:58) Focus — StudentHub";`
- Plays the Web Audio synthesizer chime and increments `sh_pomodoros.count` upon completion of focus intervals.

---

### 4.9 Automated In-Browser Test Suite (`js/tests.js`)

StudentHub contains a built-in automated test suite that runs directly inside the browser.

- Accessible on `index.html` by clicking the **"Run self-tests"** button in the footer or info panel.
- Executes 10 unit test cases verifying mathematical formulas, boundary conditions, zero-division guards, and negative value clamping.
- Displays a modal summarizing each test vector, expected output, actual result, and a pass/fail indicator.

---

### 4.10 Optional Zero-Dependency Backend (`server.js`)

While StudentHub works completely offline, `server.js` provides an optional backend for persistent multi-device access.

**Architecture:**

- Built with **Node.js standard libraries only** (`http`, `fs`, `path`, `url`).
- **RESTful Endpoints**:
  - `GET /api/subjects` / `POST /api/subjects`
  - `GET /api/budget` / `POST /api/budget`
  - `GET /api/attendance` / `POST /api/attendance`
  - `GET /api/todos` / `POST /api/todos`
  - `GET /api/pomodoros` / `POST /api/pomodoros`
  - `POST /api/backup/restore`
- **File Persistence**: Atomically writes records to `data/database.json`.
- **Static File Serving**: Serves `.html`, `.css`, `.js`, and icons with appropriate HTTP `Content-Type` headers and caching controls.

---

### 4.11 End-to-End Headless Verification (`verify_e2e.js`)

`verify_e2e.js` is a headless integration test suite executed with `node verify_e2e.js`. It runs through 8 realistic user journeys:

1. Fresh state initialization.
2. Adding 3 subjects, 4 topics, and checking study plan generation.
3. Setting an allowance of ₹5000, adding 8 expenses, and checking the 80% warning banner.
4. Enrolling in 3 courses, logging attendance, and testing the undo stack.
5. Creating 6 prioritized tasks and testing filter toggles.
6. Verifying dashboard aggregation across all metrics.
7. Simulating browser reloads to ensure persistence.
8. Verifying JSON database backup and restore operations.

---

## 5. Core Mathematical Algorithms & Complexity Analysis

| Algorithm | Function Name | Mathematical Formula | Time Complexity | Space Complexity |
|---|---|---|---|---|
| 1. Study Plan Distribution | `calculateStudyPlanDistribution` | $\left\lfloor \frac{\text{Unfinished Topics}}{\text{Days Available}} \right\rfloor$ | $O(1)$ | $O(1)$ |
| 2. Safe-to-Spend per Day | `calculateSafeToSpend` | $\max\left(0, \left\lfloor \frac{\text{Remaining Allowance}}{\text{Days Left in Month}} \right\rfloor\right)$ | $O(1)$ | $O(1)$ |
| 3. Attendance Skip Limit | `calculateClassesCanSkip` | $\max\left(0, \left\lfloor \frac{\text{Present} \times 100}{\text{Required}} - \text{Total} \right\rfloor\right)$ | $O(1)$ | $O(1)$ |
| 4. Attendance Recovery Target | `calculateClassesToAttend` | $\max\left(0, \left\lceil \frac{\text{Required} \times \text{Total} - 100 \times \text{Present}}{100 - \text{Required}} \right\rceil\right)$ | $O(1)$ | $O(1)$ |
| 5. Two-Tier Task Sorting | `sortTasksByPriorityAndDate` | Priority weight descending ( $3 > 2 > 1$ ), then Due Date ascending | $O(N \log N)$ | $O(N)$ |
| 6. Midnight Days Countdown | `calculateDaysLeft` | $\text{round}\left(\frac{\text{targetMidnight} - \text{fromMidnight}}{86,400,000}\right)$ | $O(1)$ | $O(1)$ |
| 7. Pomodoro Step Conversion | `calculatePomodoroTimerStep` | $\left\lfloor s / 60 \right\rfloor : (s \bmod 60)$ | $O(1)$ | $O(1)$ |
| 8. Expense Category Share | `calculateCategoryPercentages` | $\text{round}\left(\frac{\sum \text{Amount}_{cat}}{\text{Total Spent}} \times 100\right)$ | $O(N + C)$ | $O(C)$ |

## 6. Security, Performance & Edge-Case Handling

### 1. Cross-Site Scripting (XSS) Prevention

- In client applications that render user text via DOM templates, unescaped strings pose severe XSS vulnerabilities.
- StudentHub uses `escapeHTML(str)` in `js/common.js`:

```
function escapeHTML(str) {
  if (!str) return '';
  return String(str)
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;')
    .replace(/'/g, '&#39;');
}
```

Every user input (task titles, subject names, expense notes) is passed through this sanitizer before insertion into template strings.

### 2. Division-by-Zero Guards

- In attendance calculations: If `total === 0`, formulas guard against dividing by zero by returning `0` skips and `0` recovery classes.
- If required attendance is set to 100%, the recovery formula denominator `(100 - required)` is guarded to prevent infinite loops.
- In budget calculations: If days left in the month equals 0 (last day of the month), the divisor is clamped to `1`.

### 3. LocalStorage Quota Exceeded Protection

- Standard browser storage limits are approximately 5MB.
- `setData()` catches `QuotaExceededError` gracefully and alerts the student to export a backup rather than crashing silently.

### 4. Zero Render-Blocking Performance

- Core CSS is organized in a single stylesheet (`css/style.css`).
- System font fallbacks prevent invisible text flashes (FOIT).
- Vanilla JavaScript executes without framework reconciliation overhead, delivering 60 FPS performance even on low-end smartphones.

---

## 7. Team Presentation & Viva Q&A Guide

Use these prepared responses when presenting StudentHub to evaluators, professors, or during code defense:

### Q1: Why did your team choose Vanilla HTML/CSS/JavaScript instead of React, Next.js, or Tailwind?

> **Answer**:
> "For a college student productivity tool, our primary goals were **instant zero-latency load times, 100% offline reliability, and zero maintenance overhead**. Frameworks like React or Next.js introduce large JavaScript bundles (often over 200KB), virtual DOM diffing overhead, and complex build toolchains. By using pure Vanilla web standards, StudentHub runs on any device, loads in under 100ms, operates without an internet connection, and will continue working years from now without dependency updates or deprecation issues."

---

### Q2: How does the application store data if there is no traditional SQL or MongoDB database?

> **Answer**:
> "StudentHub uses HTML5 `localStorage` as its primary storage engine, abstracted through our `js/storage.js` layer. Each module operates against dedicated keys (`sh_subjects`, `sh_budget`, `sh_attendance`, `sh_todos`, `sh_pomodoros`). To ensure data safety, we implemented a full **JSON export and import backup system**. Additionally, for environments requiring multi-device syncing, we developed an optional zero-dependency Node.js HTTP server (`server.js`) that persists records to `data/database.json` via REST endpoints."

---

### Q3: Explain the mathematical derivation of the Attendance Recovery Formula.

> **Answer**:
> "Let $P$ be classes attended (present), $T$ be total classes held, and $R$ be the required percentage cutoff (e.g. 75). If a student attends the next $x$ consecutive classes, their new percentage will be:
>
> $$(P + x / T + x) \times 100 \geq R$$
>
> Multiplying both sides by $(T + x)$:
>
> $$100(P + x) \geq R(T + x)$$
>
> $$100P + 100x \geq RT + Rx$$
>
> Rearranging terms to isolate $x$:
>
> $$100x - Rx \geq RT - 100P$$
>
> $$x(100 - R) \geq RT - 100P$$
>
> $$x \geq (RT - 100P / 100 - R)$$
>
> Because classes are discrete integers, we apply the ceiling function:
>
> $$x = \leftft\lceil(R \cdot T - 100 \cdot P / 100 - R) \right\rceil$$
>
> If the student already has $\geq R\%$, the formula yields $\leq 0$, which we clamp to $0$."

---

### Q4: How is the Pomodoro sound synthesized without importing an audio file?

> **Answer**:
> "We utilized the browser's native **HTML5 Web Audio API**. Instead of fetching a heavy external `.mp3` file over the network, `js/common.js` creates an `AudioContext` and programs two hardware-accelerated sine-wave oscillators at **587.33 Hz** and **880.00 Hz**. We apply an exponential gain envelope that mimics a natural acoustic chime. This ensures zero network latency, zero broken audio links, and complete offline capability."

---

### Q5: How does the auto study plan adapt if a student misses study sessions?

> **Answer**:
> "The Study Plan generator in `js/planner.js` is dynamic rather than static. Instead of hardcoding topics to calendar dates, it computes:
>
> $$\text{Topics Per Day} = \left\lceil \frac{\text{Remaining Unfinished Topics}}{\text{Days Left Until Exam}} \right\rceil$$
>
> Whenever a student checks off a topic or clicks the **'Reschedule'** button, the algorithm re-queries the current date and remaining topics, dynamically redistributing the workload across the remaining timeframe."

---

## 8. Live Demonstration Script (3-Minute Walkthrough)

Follow this sequence during a live team demonstration:

| Time | Action | What to Say / Demonstrate |
|---|---|---|
| **0:00 - 0:30** | Open `index.html` **\to** Click **"Load demo data"** | *"StudentHub initializes instantly in any browser. We can pre-load a typical student semester scenario with one click. Notice how the dashboard immediately computes our next exam in red (≤ 3 days), shows daily safe spending, and flags our lowest attendance course."* |
| **0:30 - 1:00** | Navigate to **Exam Planner** (`planner.html`) | *"In the Exam Planner, subjects display remaining days and syllabus completion. If we mark a topic as done, or click 'Reschedule', our Algorithm 1 recalculates the day-wise study schedule automatically."* |
| **1:00 - 1:30** | Navigate to **Budget Tracker** (`budget.html`) | *"Here we track monthly allowances. Notice the safe-to-spend per day indicator. As we log expenses, our pure CSS stacked bar chart updates category proportions in real-time, and warning banners alert us if we cross 80% or 100% of our budget."* |
| **1:30 - 2:00** | Navigate to **Attendance Tracker** (`attendance.html`) | *"For attendance, every subject provides instant Present/Absent logging with an Undo button. Notice the smart hint cards: if we are safe, it calculates exact allowable bunks; if we are below 75%, it calculates the exact consecutive classes needed to recover."* |
| **2:00 - 2:30** | Navigate to **To-Do & Pomodoro** (`todo.html`) | *"Our task list sorts items using a two-tier algorithm—High priority first, then earliest deadline. The built-in Pomodoro timer runs 25-minute focus cycles with tab countdowns and Web Audio acoustic chimes."* |
| **2:30 - 3:00** | Click **Theme Toggle** **\to** Run **Self-Tests** | *"Finally, our dark/light theme persists across the entire application, and by clicking 'Run self-tests' on the dashboard, 10 automated unit tests execute in the browser to verify formula correctness."* |

---

*Authored by the StudentHub Engineering Team — Ready for Evaluation & Submission.*